Laboratory L3
Deadline week 7

MPI

Write an MPI-based program that computes the sum of 2 large numbers.
'large number' = a number with more than 10 digits

General considerations:
Representation of a number = an array of digits (unsigned integers — byte) in which the least significant digit is at position 0.

e.g. for the number 123 the least significant digit is 3, so the corresponding array is t[0]=3 t[1]=2 t[2]=1

The 2 large numbers are read from the files "Numar1.txt" (a number with N_1 digits) and "Numar2.txt" (a number with N_2 digits).

Each of these files contains at the beginning a number (N) which represents the number of digits, followed by the digits of the respective number.

Implementation Language > C++11.

****Variant 0** — sequential implementation**

Following versions: MPI implementations with **p** processes:

****Variant 1** (standard communication)**

IMPORTANT:
- Regardless of the MPI implementation (Microsoft, OpenMPI, MPICH, ...) the programs must be correct (without deadlock)! This means that for standard communication you must design assuming that the implementation does not use buffers — that is, MPI_Send blocks the process until the reception by the receiving process is complete. To test the correctness of the implementation — replace the standard transmission with the synchronous one MPI_Ssend.

— Following steps are required:

1) process 0

a. sets a variable *id_proces_curent*=1
b. repeats the following actions until all digits of the numbers have been read
i. reads N/(p-1) digits from the 2 files
// Attention — process 0 does not read all the digits at the beginning!!!
ii. sends them to process "*id_proces_curent*"
iii. increments "*id_proces_curent*"

2) processes with *id>0* sum the received digits (parts of the two numbers) and compute the corresponding "report" (carry);
[Constraint:] processes first receive the digits they must add and then receive the carry from the previous process!
3) each process (except the last one) sends the "report" to the next process which uses it to update the result (process id=1 does not receive a carry — it considers it equal to 0)
4) the final result is obtained in process 0, which writes the result to the file "Numar3.txt"
a. process 0
i. receives the segment of digits of the sum from each process with id>0
ii. after receiving a segment of digits from a process it writes it to the file (it does not wait to receive all digits from all the processes)
b. processes with id>1 send the digits of the sum they computed to process 0

Note: process 0 does not compute digit sums — it reads, transmits, receives and writes data.

[Optimization -!?] The addition of digits in a process with id>1 begins before waiting for the carry. Is it possible? What does this variant impose?
Variant 1.1 = Solving this potential optimization with comparative analysis of the results
=> 2 bonus points

Testing for variant 1:

1) N_1=N_2=16 ; Numar1="9999 4444 4444 9999" Numar1="9999 5555 5555 9999" with 5 processes
2) N_1=10000 and N_2=10000 (random digits) with Number of Processes: 5, 9, 17
3) N_1=100 and N_2=100000 (random digits) with Number of Processes: 5, 9, 17

****Variant 2** (scatter / gather) and process 0 involved in digits addition**

Following steps are required for this variant

1) process 0 reads the 2 numbers and stores them in 2 arrays:
> -if a number has fewer digits it is padded with non-significant digits
2) the digits of the 2 numbers are distributed to the processes using MPI_Scatter (if the condition p|N does not hold, where N=max{N_1,N_2}, N_1 = number of digits of the first number, N_2 = number of digits of the second number, then N is increased accordingly and padded with 0s)
3) the processes add the received digits and compute the "report" (carry) which then is sent to the next process (except the last process which does not send a carry).
4) the final result is obtained in process 0 (MPI_Gather is used).
5) process 0 writes the result to the file "Numar3.txt"

Testing for variant 2:

1) N_1=N_2=16 ; Numar1="9999 4444 4444 9999" Numar1="9999 5555 5555 9999" with 4 processe
2) N_1=1000 and N_2=1000 (random digits) with Number of Processes: 4, 8, 16
3) N_1=100 and N_2=100000 (random digits) with Number of Processes: 4, 8, 16

****Variant 3** — asynchronous communication**

Following steps are required:
1) process 0
- sets a variable *id_proces_curent*=1
- repeats the following actions until all digits of the numbers have been read
    a. reads N/p digits from the 2 files
    // Attention — process 0 does not read all the digits at the beginning!
    b. sends them to process "*id_proces_curent*" using MPI_Isend
    c. increments "*id_proces_curent*"
2) a process with id<>0 receives the set of digits from process 0 and performs the addition into a result vector and updates the "report" (carry) which it sends to the next process
!!!(note: a process with id>1 receives information from process 0 and from process (id-1) but the order between the 2 is not guaranteed ... you are required to use **MPI_Irecv**)
3) the final result is obtained in process 0 by aggregating the results of all the other processes; the aggregation will be done using asynchronous transmission!!!
4) process 0 writes the result to the file "Numar3.txt"

Note: process 0 does not compute digit sums — it reads, transmits, receives and writes data.

Testing for variant 3:

1) N_1=N_2=16 ; Numar1="9999 4444 4444 9999" Numar1="9999 5555 5555 9999" with 5 processes
2) N_1=10000 and N_2=10000 (random digits) with Number of Processes: 5, 9, 17
3) N_1=100 and N_2=100000 (random digits) with Number of Processes: 5, 9, 17

Obs:

For all variants Include the reading of the numbers in the execution time!

Do not forget to verify correctness by comparing the result files obtained in the sequential variant and in the parallel variants for each of the 10 executions.